

664. Strange Printer

1. Я сократил все одинаковые подряд идущие буквы до одной буквы (aaa => a) т.к. это не влияет на условия задачи и ответ, но избавляет от избыточных данных - дпшка посчитается быстрее.

2. Для строки длины n строим дп n на n + 1.

```
vector<vector<int>> dp(n, vector<int>(n + 1, -1));  
return deep(1s, dp, 0, n - 1);
```

И запускаем на отрезке [0, n - 1]

```
if (r < l) return 0;  
if (dp[l][r] != -1) return dp[l][r];
```

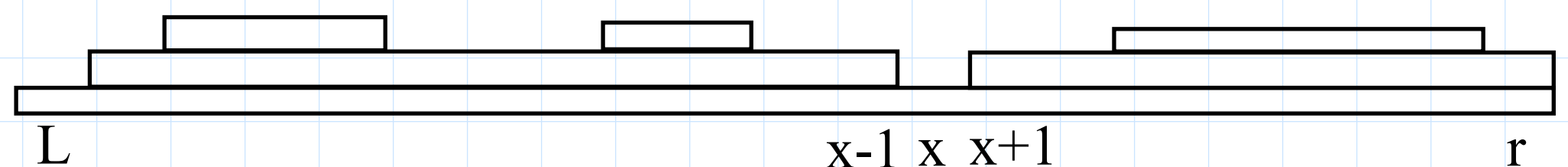
Если на отрезке ответа не может быть или он уже был найден - возвращаем

```
dp[l][r] = deep(s, dp, l + 1, r) + 1;
```

Предварительный ответ на отрезке считаем, как ответ отрезка меньшей длины (на 1) + 1 т.к. рядом стоящие буквы различны из-за предобработки на шаге 1

```
for (int x = l + 1; x <= r; ++x) {  
    if (s[l] == s[x]) {  
        dp[l][r] = min(dp[l][r], deep(s, dp, l, x - 1) + deep(s, dp, x + 1, r));  
    }  
}
```

Тут проще визуализировать: представим слои печати принтера =>



Будем искать букву, которая может быть на том-же слое что и буква на месте L, пересчитываем результат и сравниваем с предыдущим оптимальным.

Проводим дпшку "в глубину" и получаем оптимальный результат для печати на отрезке.



719. Find K-th Smallest Pair Distance

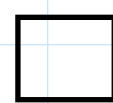
Заметим, что ответ будет целым неотрицательным числом, поэтому сделаем бинпоиск по ответу, где $l = 0$, $r = \max[\text{dist}] + 1$

```
int smallestDistancePair(vector<int>& nums_raw, int k) {
    vector<int> nums = nums_raw;
    sort(nums.begin(), nums.end());
    int l = 0;
    int r = nums.back() - nums[0] + 1;
    while (r - l > 1) {
        int m = (r + l) / 2;
        int cur = compute(nums, m);
        if (cur < k) {
            l = m;
        } else {
            r = m;
        }
    }
    return l;
}
```

Теперь надо посчитать сколько dist меньше этого ответа.

```
int compute(vector<int>& nums, int max_dif) {
    int amount = 0;
    int n = nums.size();
    for (int i = 0; i < n; i++) {
        int l = i;
        int r = n;
        while (r - l > 1) {
            int m = (r + l) / 2;
            if (nums[m] - nums[i] < max_dif) {
                l = m;
            } else {
                r = m;
            }
        }
        amount += l - i;
    }
    return amount;
}
```

Надо просуммировать dist-ы для каждого i (i как левой границы), бинпоиском находя правую границу по выбранному основному ответу



502. IPO

Сначала возьмём пары капитал-профит и отсортируем по необходимому капиталу.

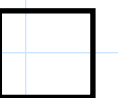
```
vector<pair<int, int>> arr(profits.size());
for (int i = 0; i != profits.size(); ++i) {
    arr[i] = {capital[i], profits[i]};
}
sort(arr.begin(), arr.end());
arr.push_back({INT_MAX, INT_MAX});
```

```
priority_queue<int> heap;
heap.push(0);
int j = 0, answer = w, n = profits.size();
for (int i = 0; i != k; ++i) {
    while (j < n && arr[j].first <= answer) {
        heap.push(arr[j++].second);
    }
    if (heap.top() < 1) {
        return answer;
    } else {
        answer += heap.top();
        heap.pop();
    }
}
return answer;
```

Создаём кучу для профитов

И на каждой из k итераций:

1. Добираем кучу профитов, если хватает капитала
2. Берём из кучи наибольший профит и добавляем к капиталу



140. Word Break II

Для решения этой задачи я решил использовать бор, его реализация будет на нодах

```
struct Node {
    map<char, Node*> links;
    Node () {}
};
```

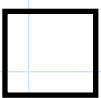
Инициализируем бор из имеющихся слов, конец слова помечаем ' ' с переходом в корень

```
Node *tree = new Node();
Node *cur;
for (string line : wordDict) {
    cur = tree;
    for (char el : line) {
        if (cur->links.count(el) == 0) cur->links[el] = new Node();
        cur = cur->links[el];
    }
    cur->links[' '] = tree;
}
```

Дальше собираем последовательности слов, проходя по бору в глубину

```
if (n == s.size() && node->links.count(' ') == 1) { answer.push_back(sousage); return; }
for (auto const [key, val] : node->links) {
    sousage.push_back(key);
    if (key == ' ') {
        deep(n, s, val);
    } else if (s[n] == key) {
        deep(n + 1, s, val);
    }
    sousage.pop_back();
}
```

Т.к. концы слов ведут в начало, можно непрерывно продолжать обход



330. Patching Array

Тут следует начать с доказательства:

Б.И.: Единица не получится без единицы, поэтому единица должна быть всегда.

П.И.: Для i -го числа, все числа до него можно получить суммой чисел из массива.

Ш.И.: Если i -е число не получается из чисел массива меньше i и i в массиве нет, оптимально будет добавить i в массив, это по сути единственное решение, без которого дальше ничего не получится. Заметим, что все числа до $2i - 1$ можно будет собрать из чисел массива. Заметим также, что если в массиве есть j , такое что $i < j < 2i - 1$, то все числа до $2i - 1 + j$ тоже могут быть получены из чисел массива.

```
long long i = 1, s = 0, j = 0, b = 0;
int answer = 0;
while (i <= n) {
    while (j < nums.size()) {
        if (nums[j] <= i) { s += nums[j]; ++j; }
        else break;
    }
    if (s < i) {
        ++answer;
        s += i;
        i = s;
    } else {
        ++i;
    }
}
return answer;
```

Исходя из замечаний к шагу индукции, я буду суммировать числа из массива меньше i , если сумма будет меньше чем i , добавлю i к массиву (условно в рамках кода) и сделаю прыжок на сумму $+ i$.

